

Memory Organization and Memory Technology

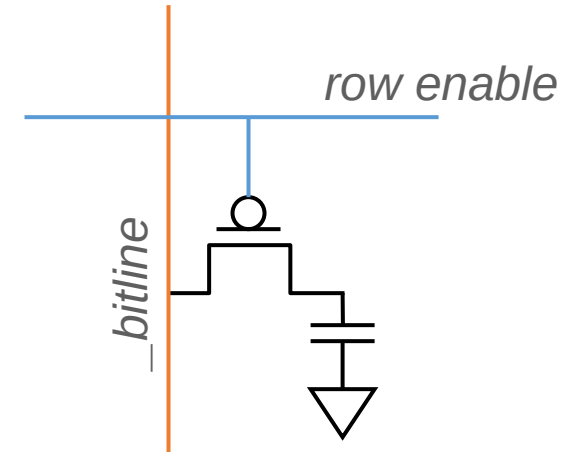
Readings

- *Digital Design and Computer Architecture – David Harris & Sarah Harris*
 - Chapter 8

Memory Technology: DRAM and SRAM

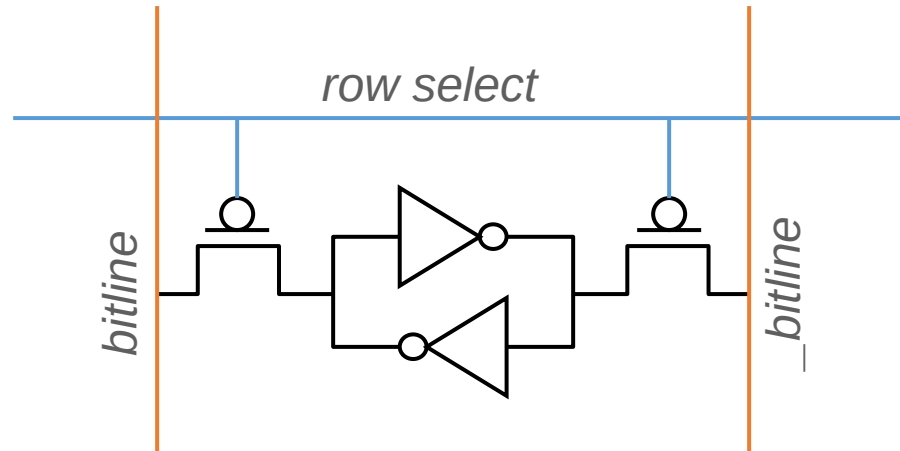
Memory Technology: DRAM

- Dynamic random access memory
- Capacitor charge state indicates stored value
 - Whether the capacitor is charged or discharged indicates storage of 1 or 0
 - 1 capacitor
 - 1 access transistor
- Capacitor leaks through the RC path
 - DRAM cell loses charge over time
 - DRAM cell needs to be refreshed



Memory Technology: SRAM

- Static random access memory
- Two cross coupled inverters store a single bit
 - Feedback path enables the stored value to persist in the “cell”
 - 4 transistors for storage
 - 2 transistors for access



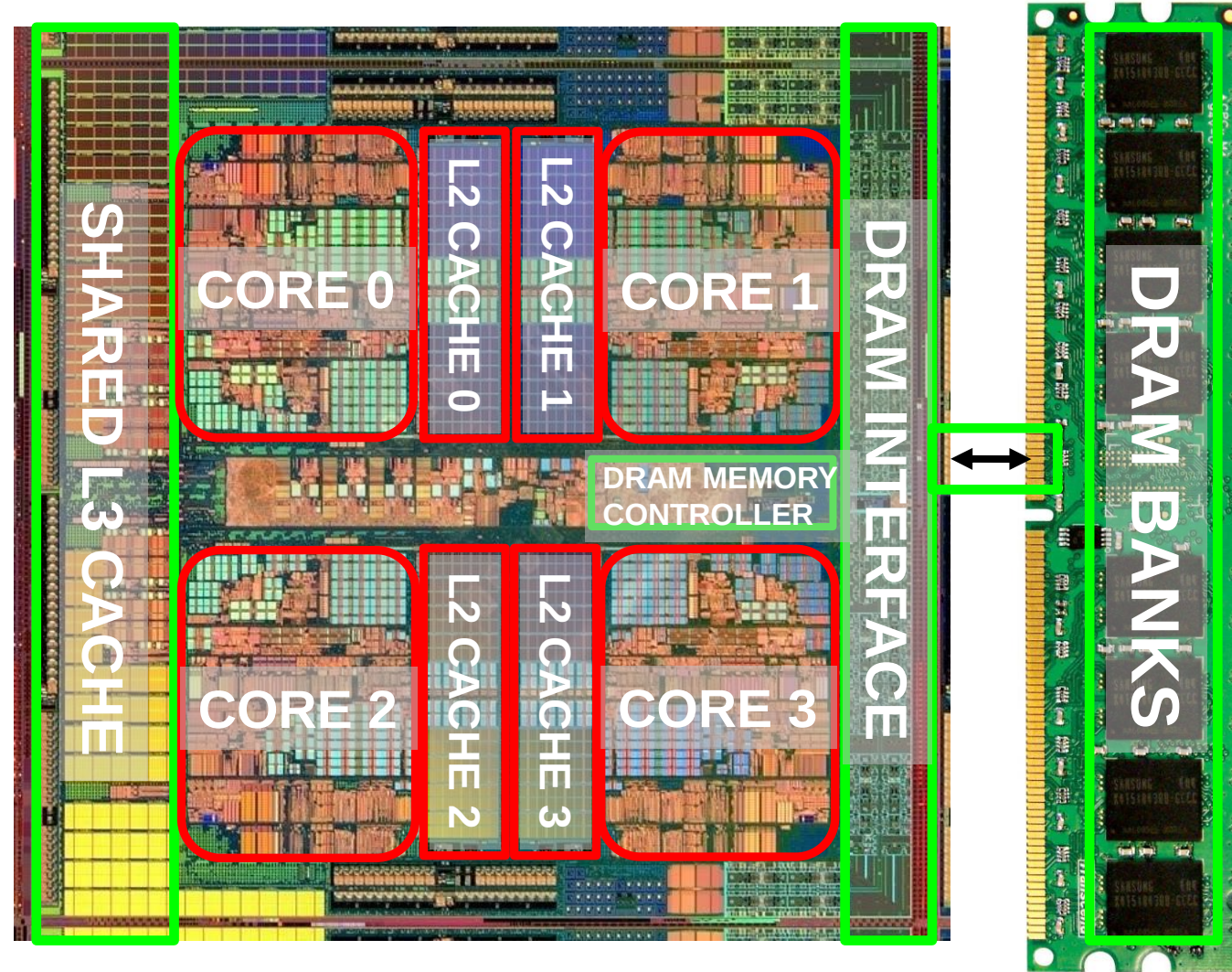
DRAM vs. SRAM

- DRAM
 - Slower access (capacitor)
 - Higher density (1T 1C cell)
 - Lower cost
 - Requires refresh (power, performance, circuitry)
 - Manufacturing requires putting capacitor and logic together
- SRAM
 - Faster access (no capacitor)
 - Lower density (6T cell)
 - Higher cost
 - No need for refresh
 - Manufacturing compatible with logic process (no capacitor)

Memory Hierarchy and Caches

The Memory Hierarchy

Memory in a Modern System



Ideal Memory

- Zero access time (latency)
- Infinite capacity
- Zero cost
- Infinite bandwidth (to support multiple accesses in parallel)

The Problem

- Ideal memory's requirements oppose each other
- Bigger is slower
 - Bigger → Takes longer to determine the location
- Faster is more expensive
 - Memory technology: SRAM vs. DRAM vs. Disk vs. Tape
- Higher bandwidth is more expensive
 - Need more banks, higher frequency, or faster technology

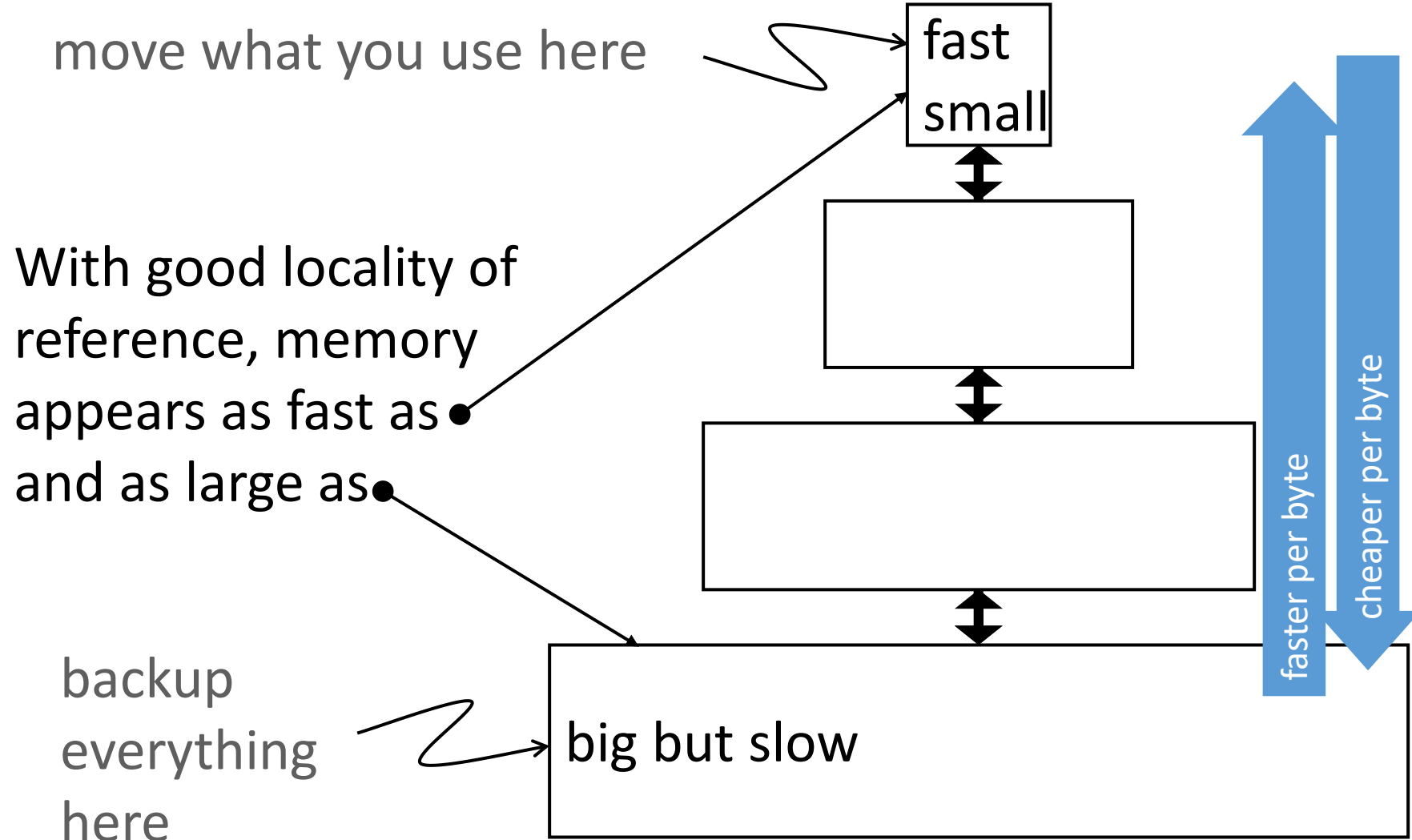
The Problem

- Bigger is slower
 - SRAM, 512 Bytes, sub-nanosec
 - SRAM, KByte~MByte, ~nanosec
 - DRAM, Gigabyte, ~50 nanosec
 - Hard Disk, Terabyte, ~10 millisec
- Faster is more expensive (dollars and chip area)
 - SRAM, < 10\$ per Megabyte
 - DRAM, < 1\$ per Megabyte
 - Hard Disk < 1\$ per Gigabyte
 - These sample values (circa ~2011) scale with time

Why Memory Hierarchy?

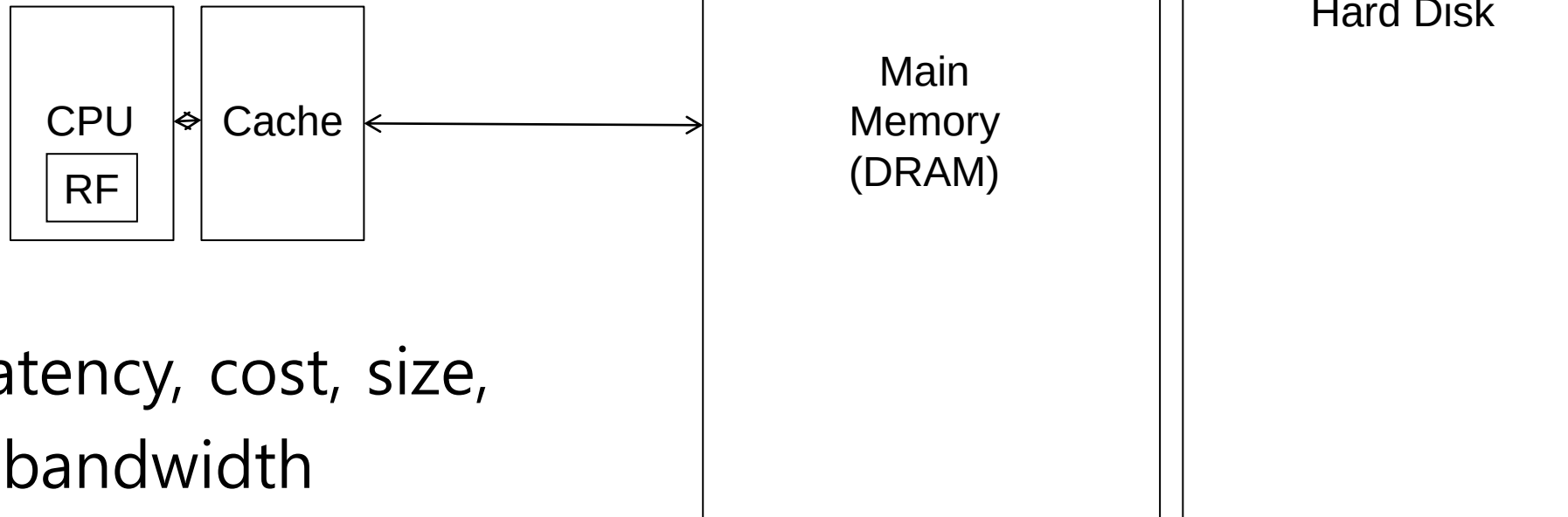
- We want both fast and large
- But we cannot achieve both with a single level of memory
- Idea: Have multiple levels of storage (progressively bigger and slower as the levels are farther from the processor) and ensure most of the data the processor needs is kept in the fast(er) level(s)

The Memory Hierarchy



Memory Hierarchy

- Fundamental tradeoff
 - Fast memory: small
 - Large memory: slow
- Idea: **Memory hierarchy**



- Latency, cost, size, bandwidth

Locality

- One's recent past is a very good predictor of his/her near future.
- **Temporal Locality**: If you just did something, it is very likely that you will do the same thing again soon
 - since you are here today, there is a good chance you will be here again and again regularly
- **Spatial Locality**: If you did something, it is very likely you will do something similar/related (in space)
 - every time I find you in this room, you are probably sitting close to the same people

Memory Locality

- A “typical” program has a lot of locality in memory references
 - typical programs are composed of “loops”
- **Temporal**: A program tends to reference the same memory location many times and all within a small window of time
- **Spatial**: A program tends to reference a cluster of memory locations at a time
 - most notable examples:
 - 1. instruction memory references
 - 2. array/data structure references

Caching Basics: Exploit Temporal Locality

- Idea: Store recently accessed data in automatically managed fast memory (called cache)
- Anticipation: the data will be accessed again soon
- Temporal locality principle
 - Recently accessed data will be again accessed in the near future
 - This is what Maurice Wilkes had in mind:
 - Wilkes, “Slave Memories and Dynamic Storage Allocation,” IEEE Trans. On Electronic Computers, 1965.
 - “The use is discussed of a fast core memory of, say 32000 words as a slave to a slower core memory of, say, one million words in such a way that in practical cases the effective access time is nearer that of the fast memory than that of the slow memory.”

Caching Basics: Exploit Spatial Locality

- Idea: Store addresses adjacent to the recently accessed one in automatically managed fast memory
 - Logically divide memory into equal size blocks
 - Fetch to cache the accessed block in its entirety
- Anticipation: nearby data will be accessed soon
- Spatial locality principle
 - Nearby data in memory will be accessed in the near future
 - E.g., sequential instruction access, array traversal
 - This is what IBM 360/85 implemented
 - 16 Kbyte cache with 64 byte blocks
 - Liptay, “Structural aspects of the System/360 Model 85 II: the cache,” IBM Systems Journal, 1968.

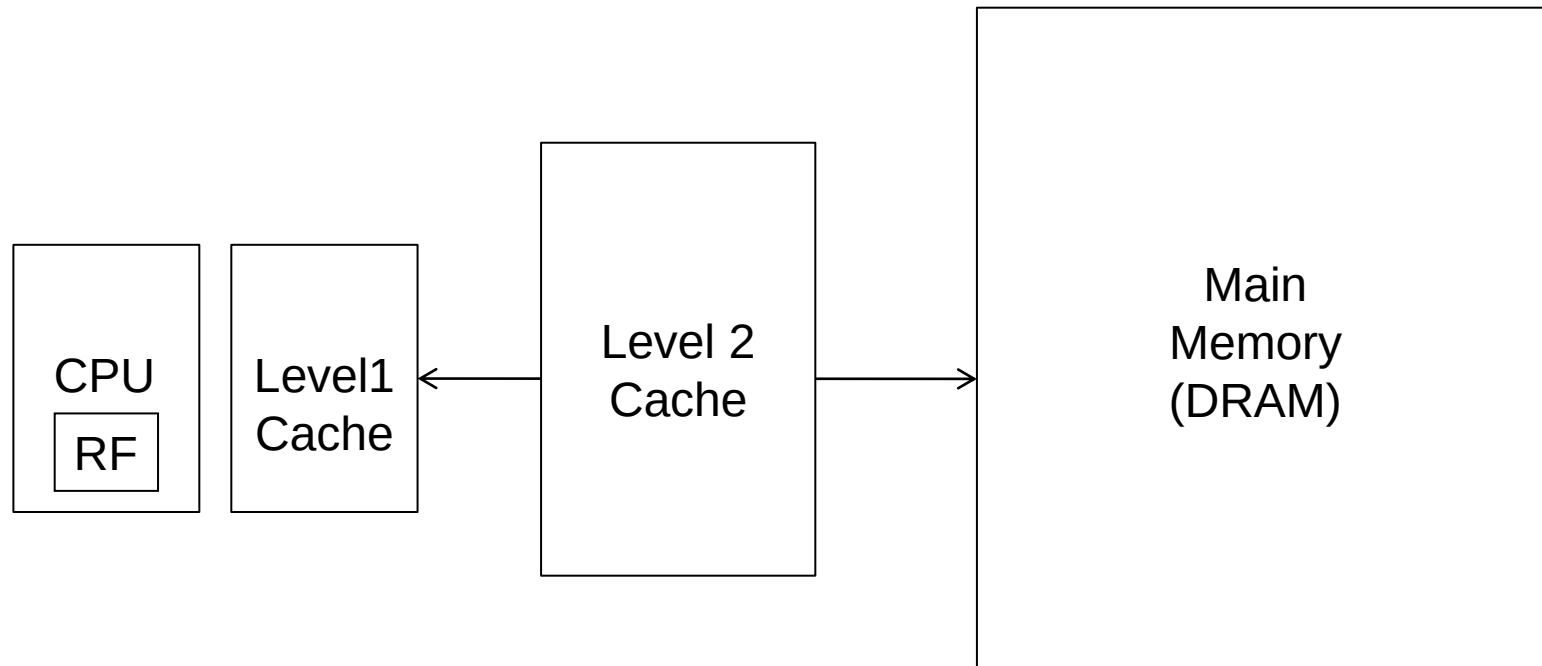
The Bookshelf Analogy

- Book in your hand
- Desk
- Bookshelf
- Boxes at home
- Boxes in storage

- Recently-used books tend to stay on desk
 - Comp Arch books, books for classes you are currently taking
 - Until the desk gets full
- Adjacent books in the shelf needed around the same time
 - If I have organized/categorized my books well in the shelf

Caching in a Pipelined Design

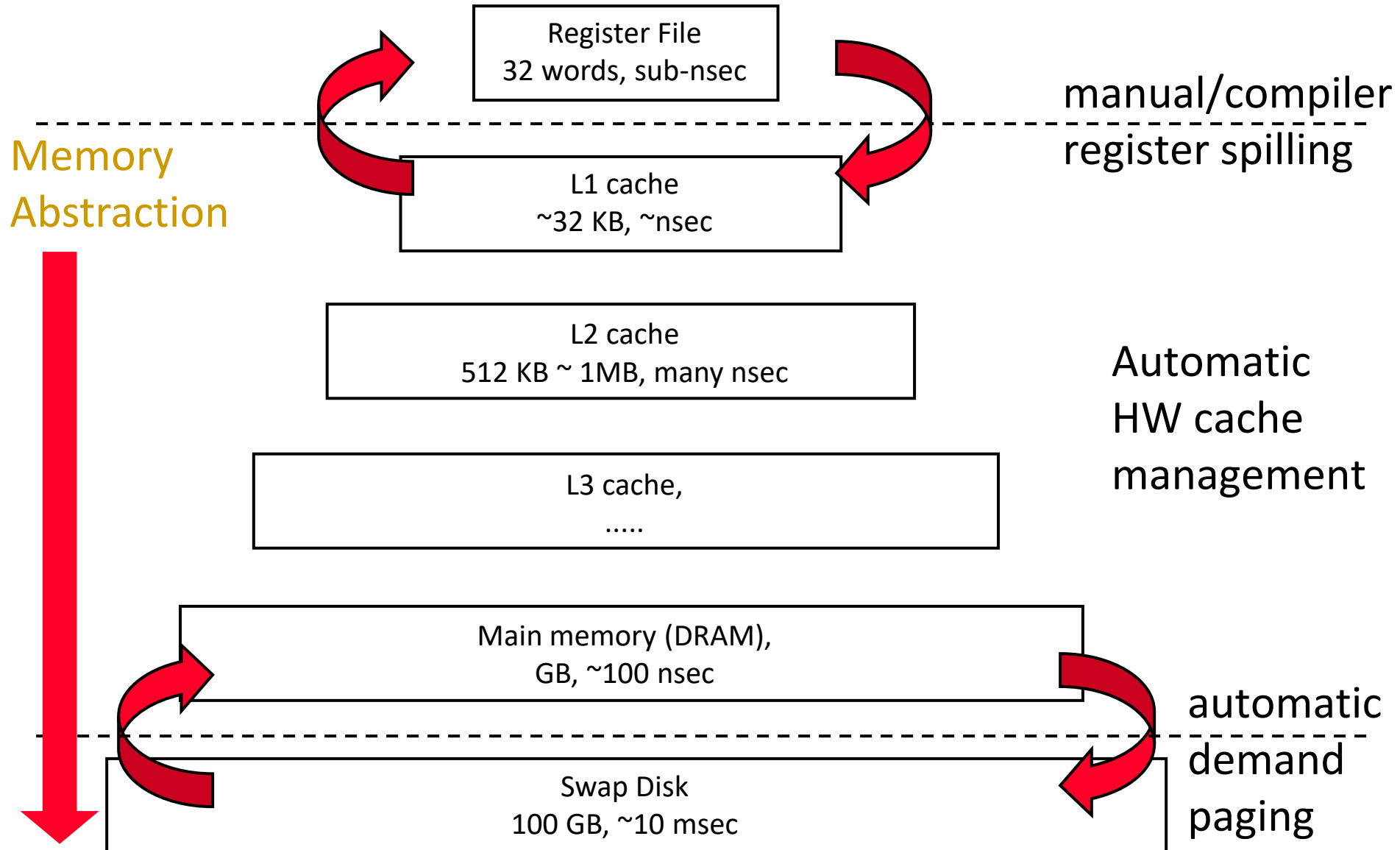
- The cache needs to be tightly integrated into the pipeline
 - Ideally, access in 1-cycle so that load-dependent operations do not stall
- High frequency pipeline → Cannot make the cache large
 - But, we want a large cache AND a pipelined design
- Idea: **Cache hierarchy**



A Note on Manual vs. Automatic Management

- **Manual:** Programmer manages data movement across levels
 - too painful for programmers on substantial programs
 - still done in some embedded processors (on-chip scratch pad SRAM in lieu of a cache) and GPUs (called "shared memory")
- **Automatic:** Hardware manages data movement across levels, transparently to the programmer
 - ++ programmer's life is easier
 - the average programmer doesn't need to know about it
 - You don't need to know how big the cache is and how it works to write a "correct" program! (What if you want a "fast" program?)

A Modern Memory Hierarchy



Hierarchical Latency Analysis

- For a given memory hierarchy level i it has a technology-intrinsic access time of t_i . The perceived access time T_i is longer than t_i
- Except for the outer-most hierarchy, when looking for a given address there is
 - a chance (hit-rate h_i) you “hit” and access time is t_i
 - a chance (miss-rate m_i) you “miss” and access time $t_i + T_{i+1}$
 - $h_i + m_i = 1$
- Thus

$$T_i = h_i \cdot t_i + m_i \cdot (t_i + T_{i+1})$$

$$T_i = t_i + m_i \cdot T_{i+1}$$

h_i and m_i are defined to be the hit-rate and miss-rate of just the references that missed at L_{i-1}

Hierarchy Design Considerations

- Recursive latency equation

$$T_i = t_i + m_i \cdot T_{i+1}$$

- The goal: achieve desired T_1 within allowed cost
- $T_i \approx t_i$ is desirable
- Keep m_i low
 - increasing capacity C_i lowers m_i , but beware of increasing t_i
 - lower m_i by smarter cache management (replacement::anticipate what you don't need, prefetching::anticipate what you will need)
- Keep T_{i+1} low
 - faster lower hierarchies, but beware of increasing cost
 - introduce intermediate hierarchies as a compromise

Intel Pentium 4 Example

- P4, 3.6 GHz
 - L1 cache
 - $C_1 = 16K$
 - $t_1 = 4$ cyc int / 9 cycle fp
 - L2 cache
 - $C_2 = 1024$ KB
 - $t_2 = 18$ cyc int / 18 cyc fp
 - Main memory
 - $t_3 = \sim 50ns$ or 180 cyc
 - Notice
 - best case latency is not 1
 - worst case access latencies are into 200+ cycles
- if $m_1=0.1, m_2=0.1$
 $T_1=7.6, T_2=36$
- if $m_1=0.01, m_2=0.01$
 $T_1=4.2, T_2=19.8$
- if $m_1=0.05, m_2=0.01$
 $T_1=5.00, T_2=19.8$
- if $m_1=0.01, m_2=0.50$
 $T_1=5.08, T_2=108$

Cache Basics and Operation

Readings

- *Digital Design and Computer Architecture – David Harris & Sarah Harris*
 - Chapter 8

Caching Basics

- **Block (line):** Unit of storage in the cache

- Memory is logically divided into cache blocks that map to locations in the cache

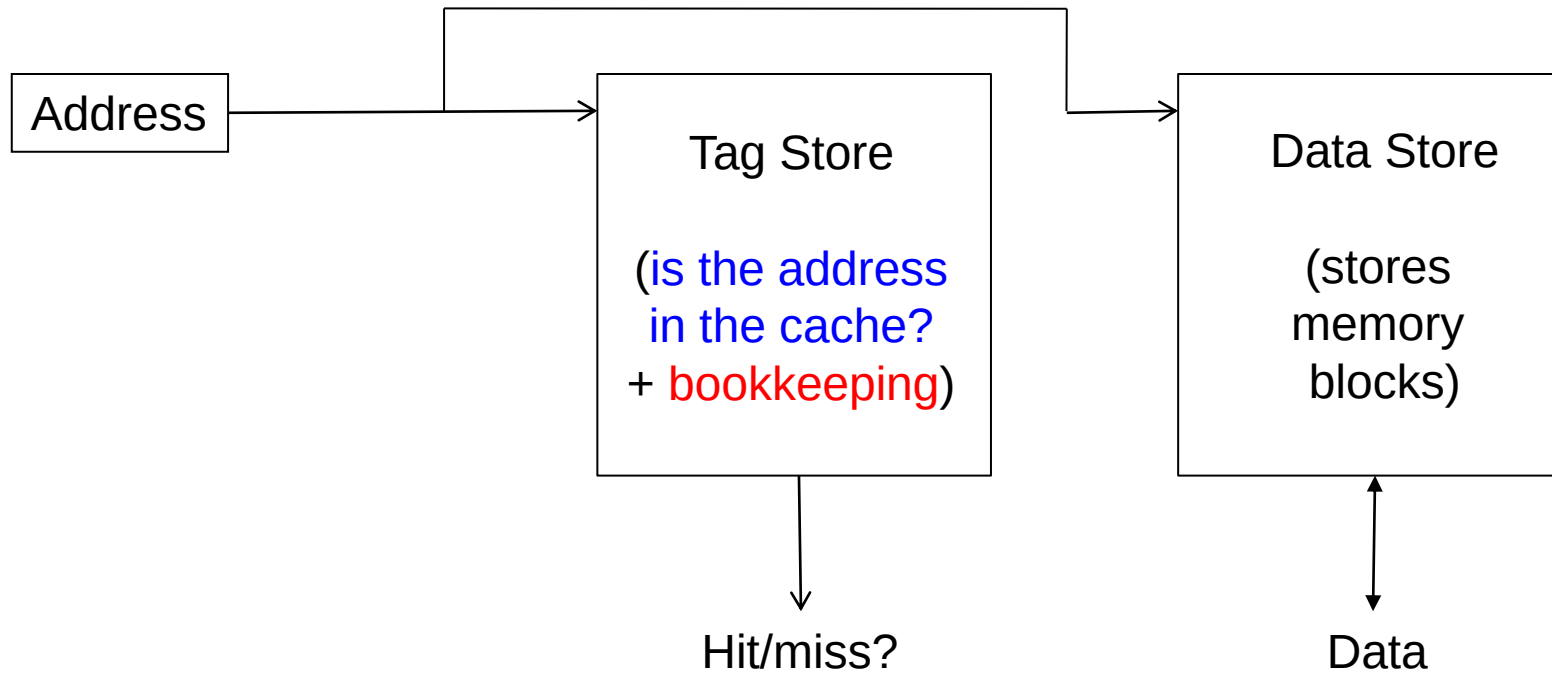
- **On a reference:**

- **HIT:** If in cache, use cached data instead of accessing memory
 - **MISS:** If not in cache, bring block into cache
 - Maybe have to kick something else out to do it

- **Some important cache design decisions**

- **Placement:** where and how to place/find a block in cache?
 - **Replacement:** what data to remove to make room in cache?
 - **Granularity of management:** large or small blocks? Subblocks?
 - **Write policy:** what do we do about writes?
 - **Instructions/data:** do we treat them separately?

Cache Abstraction and Metrics



- Cache hit rate = $(\# \text{ hits}) / (\# \text{ hits} + \# \text{ misses}) = (\# \text{ hits}) / (\# \text{ accesses})$
- Average memory access time (AMAT)
= $(\text{hit-rate} * \text{hit-latency}) + (\text{miss-rate} * \text{miss-latency})$

A Basic Hardware Cache Design

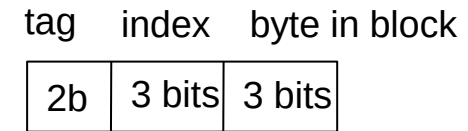
- We will start with a basic hardware cache design
- Then, we will examine a multitude of ideas to make it better

Blocks and Addressing the Cache

- Memory is logically divided into fixed-size blocks

- Each block maps to a location in the cache, determined by the **index bits** in the address

- used to index into the tag and data stores



8-bit address

- Cache access:

- 1) index into the tag and data stores with index bits in address
- 2) check valid bit in tag store
- 3) compare tag bits in address with the stored tag in tag store

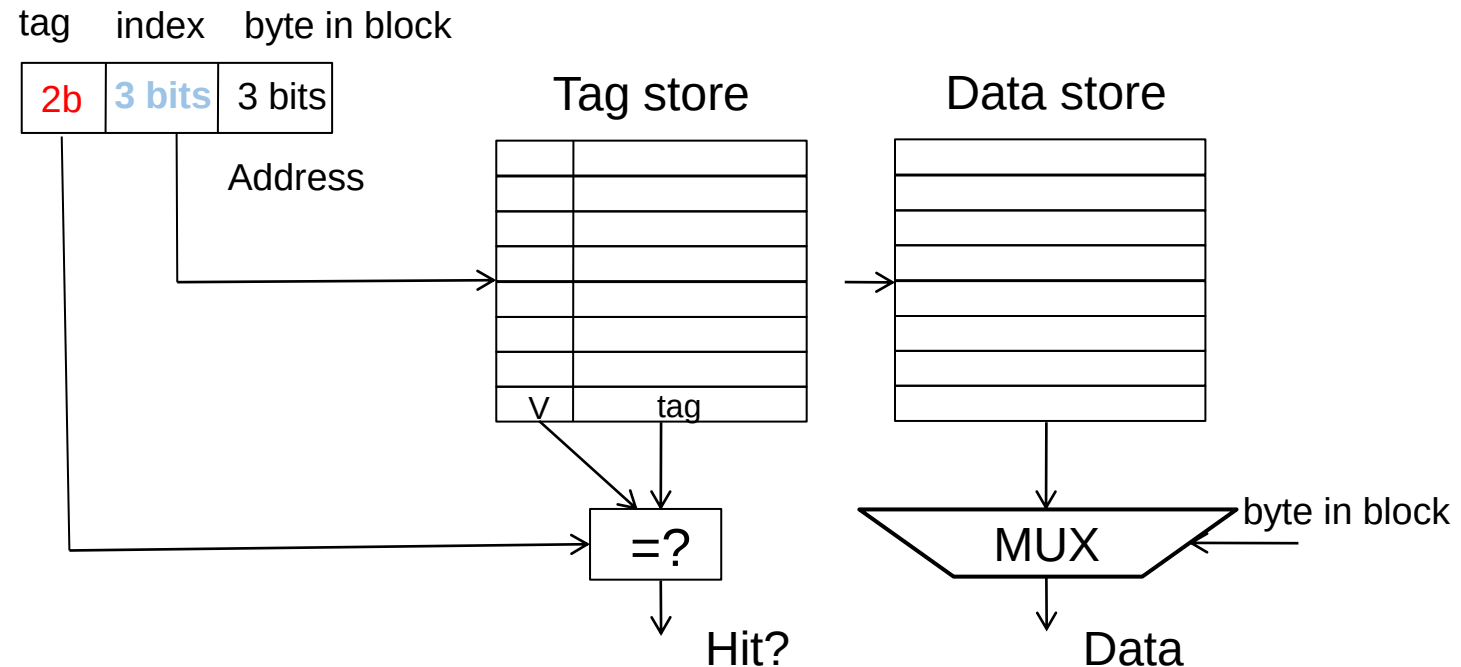
- If a block is in the cache (cache hit), **the stored tag should be valid and match the tag of the block**

Direct-Mapped Cache: Placement and Access

Block: 00000
Block: 00001
Block: 00010
Block: 00011
Block: 00100
Block: 00101
Block: 00110
Block: 00111
Block: 01000
Block: 01001
Block: 01010
Block: 01011
Block: 01100
Block: 01101
Block: 01110
Block: 01111
Block: 10000
Block: 10001
Block: 10010
Block: 10011
Block: 10100
Block: 10101
Block: 10110
Block: 10111
Block: 11000
Block: 11001
Block: 11010
Block: 11011
Block: 11100
Block: 11101
Block: 11110
Block: 11111

Main memory

- Assume byte-addressable memory: 256 bytes, 8-byte blocks
→ 32 blocks
- Assume cache: 64 bytes, 8 blocks
 - Direct-mapped: A block can go to only one location
 - Addresses with same index contend for the same location
 - Cause conflict misses

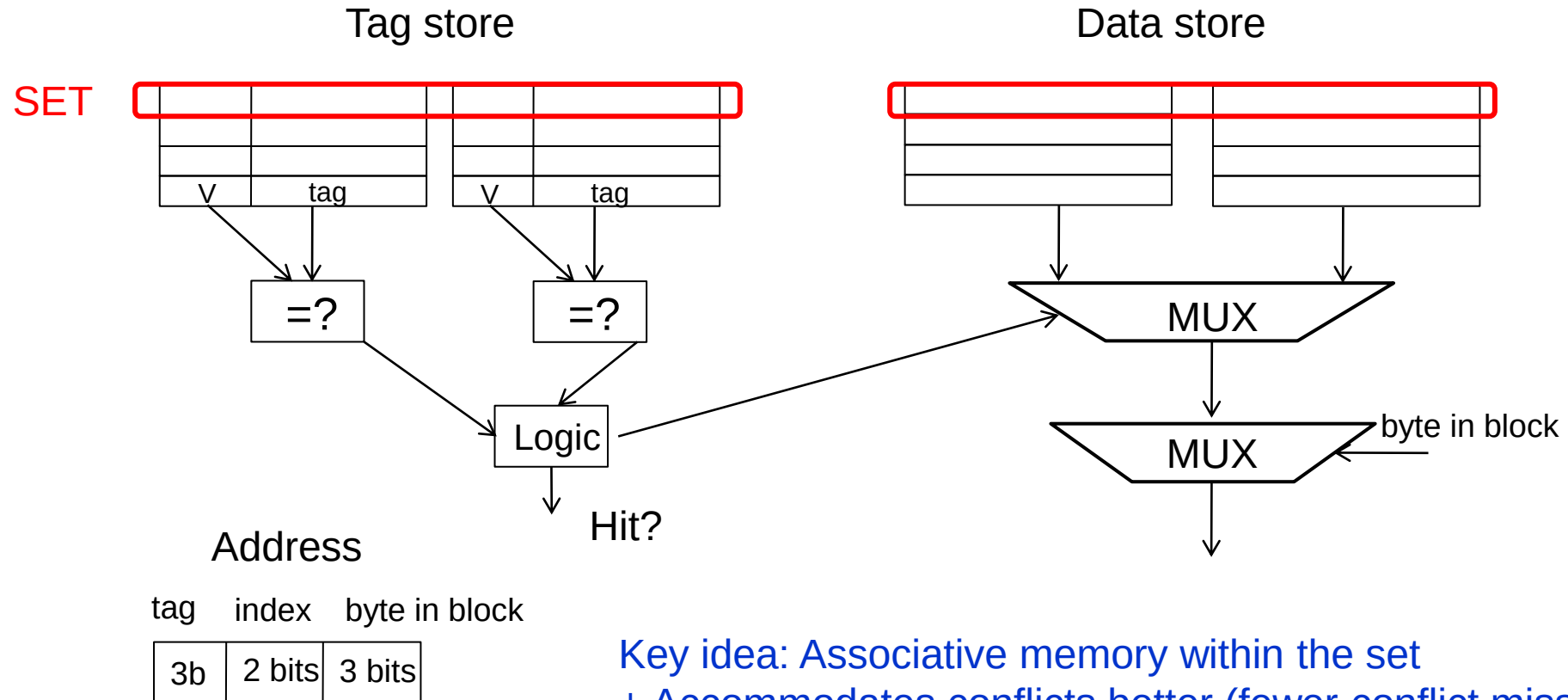


Direct-Mapped Caches

- **Direct-mapped cache:** Two blocks in memory that map to the same index in the cache cannot be present in the cache at the same time
 - One index $\rightarrow$ one entry
- Can lead to 0% hit rate if more than one block accessed in an interleaved manner map to the same index
 - Assume addresses A and B have the same index bits but different tag bits
 - A, B, A, B, A, B, A, B, ... $\rightarrow$ conflict in the cache index
 - All accesses are **conflict misses**

Set Associativity

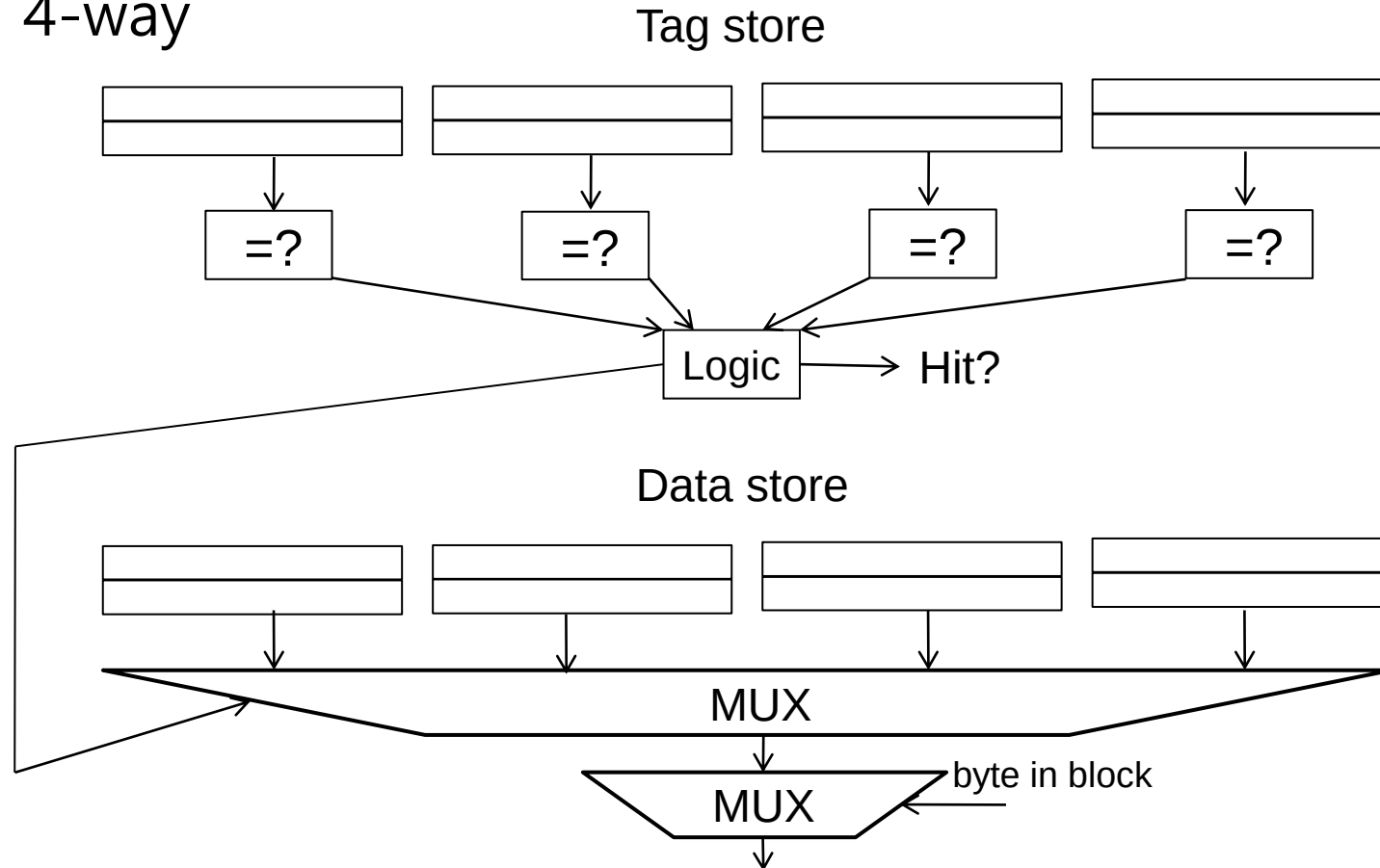
- Addresses 0 and 8 always conflict in direct mapped cache
- Instead of having one column of 8, have 2 columns of 4 blocks



Key idea: Associative memory within the set
+ Accommodates conflicts better (fewer conflict misses)
-- More complex, slower access, larger tag store

Higher Associativity

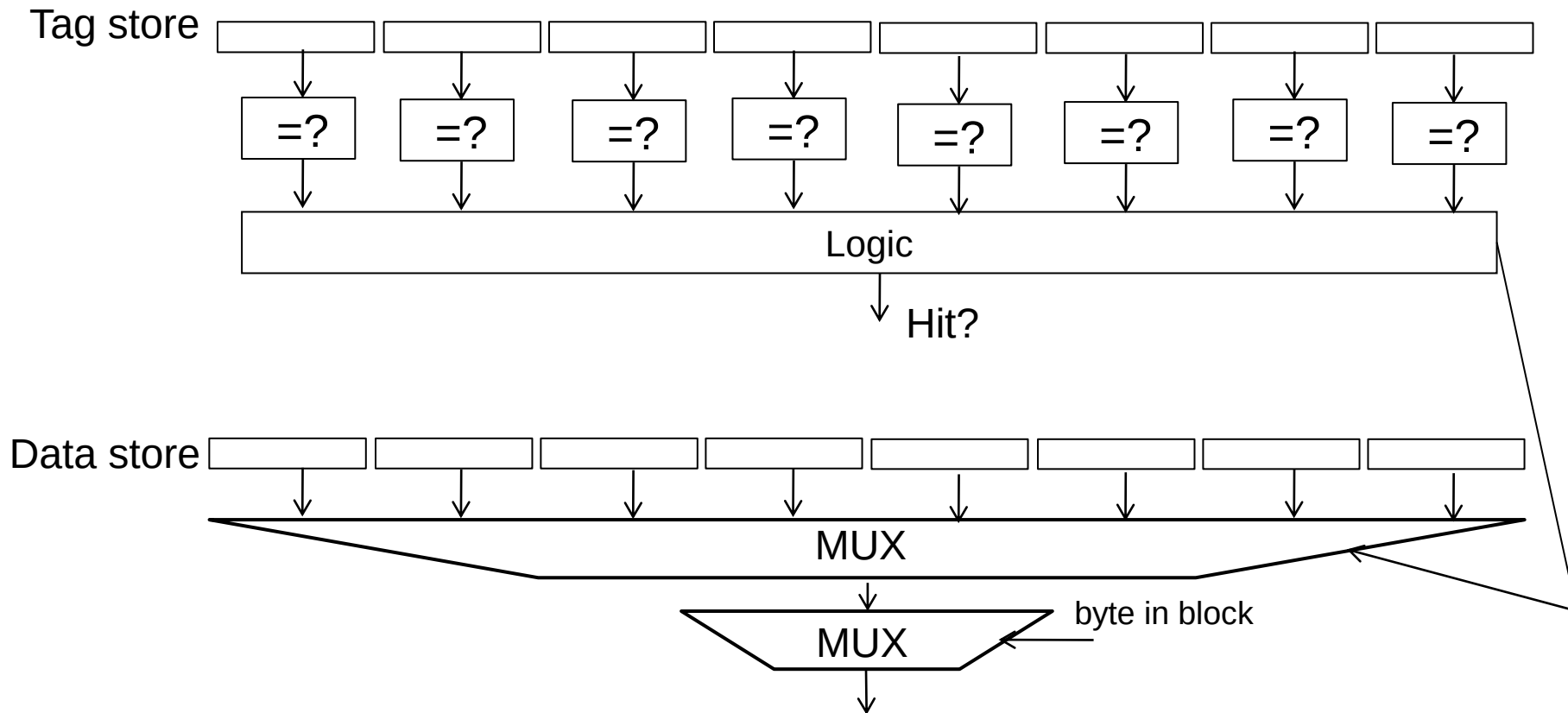
- 4-way



- + Likelihood of conflict misses even lower
- More tag comparators and wider data mux; larger tags

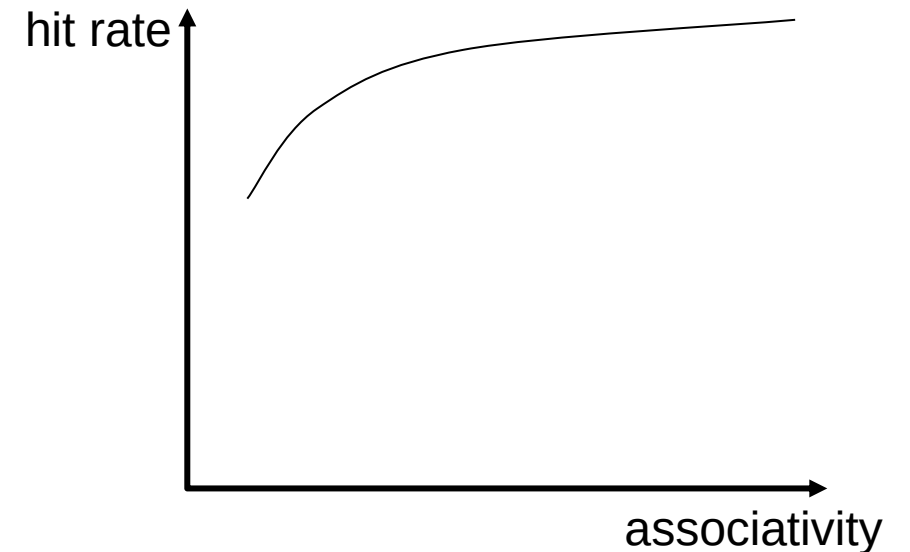
Full Associativity

- Fully associative cache
 - A block can be placed in **any** cache location



Associativity (and Tradeoffs)

- **Degree of associativity**: How many blocks can map to the same index (or set)?
- Higher associativity
 - ++ Higher hit rate
 - Slower cache access time (hit latency and data access latency)
 - More expensive hardware (more comparators)
- Diminishing returns from higher associativity



Cache Examples

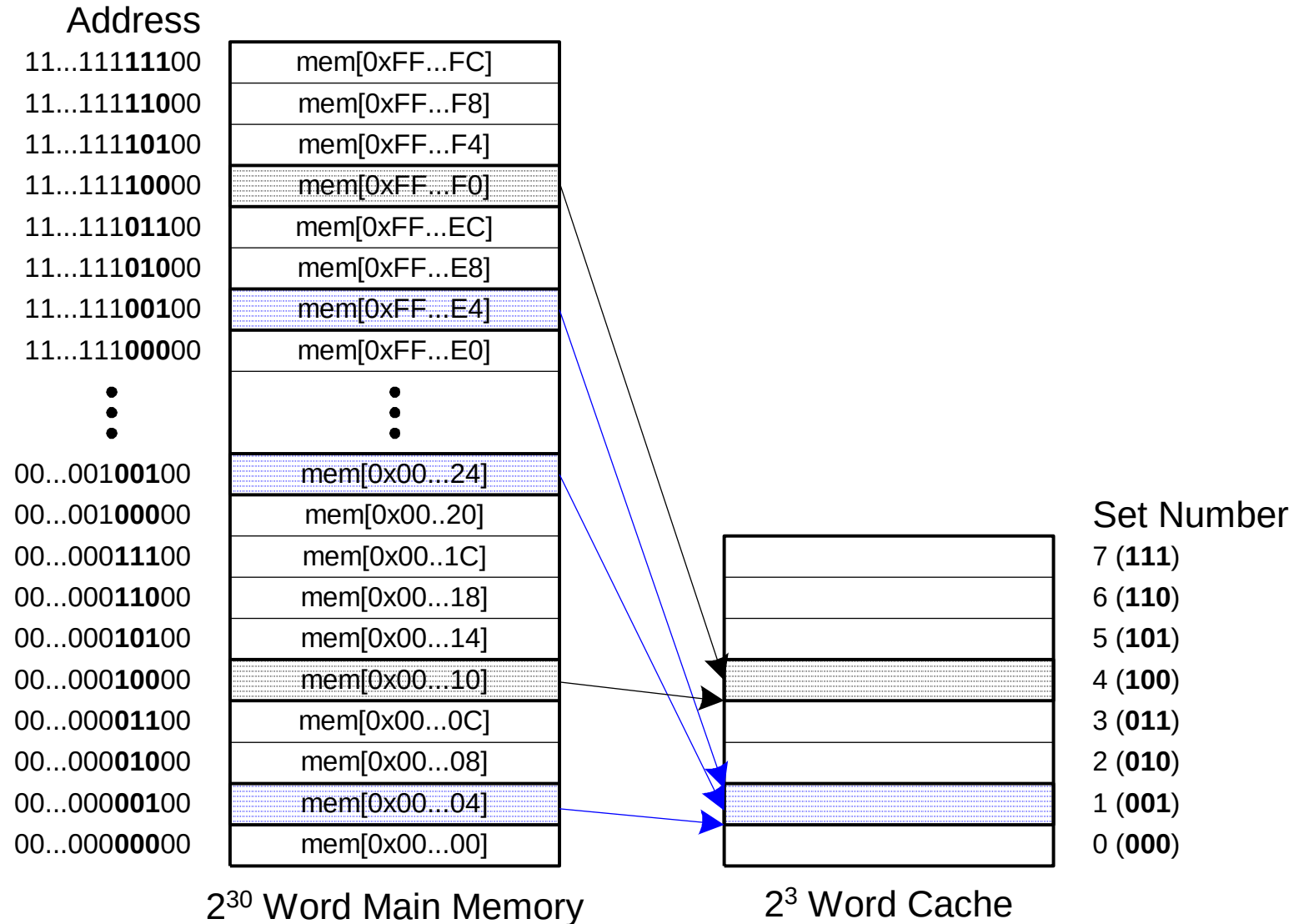
Cache Terminology

- Capacity (C):
 - the number of data bytes a cache stores
- Block size (b):
 - bytes of data brought into cache at once
- Number of blocks ($B = C/b$):
 - number of blocks in cache: $B = C/b$
- Degree of associativity (M):
 - number of blocks in a set
- Number of sets ($S = B/M$):
 - each memory address maps to exactly one cache set

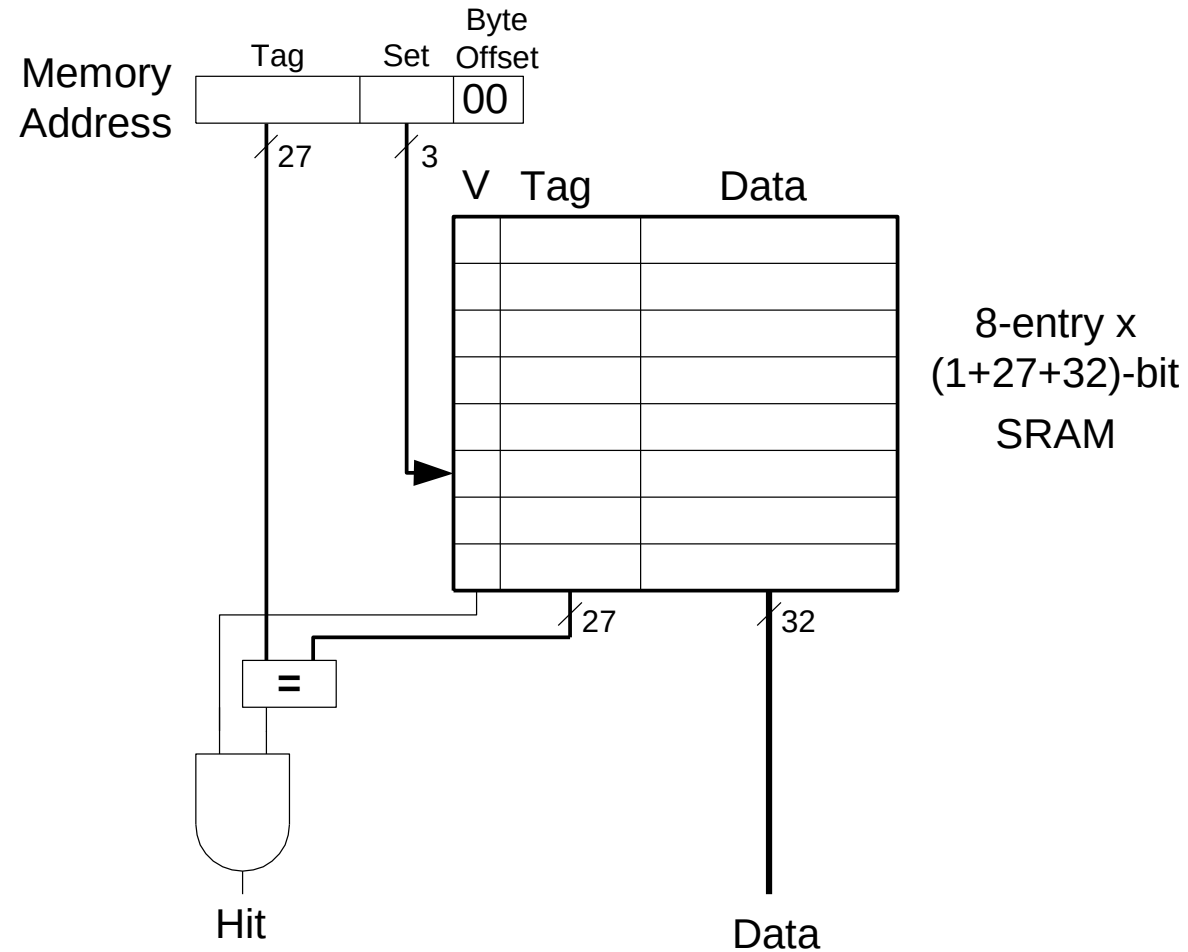
How is data found?

- Cache organized into S sets
- Each memory address maps to exactly one set
- Caches categorized by number of blocks in a set:
 - Direct mapped: 1 block per set
 - N-way set associative: N blocks per set
 - Fully associative: all cache blocks are in a single set
- Examine each organization for a cache with:
 - Capacity ($C = 8$ words)
 - Block size ($b = 1$ word)
 - So, number of blocks ($B = 8$)

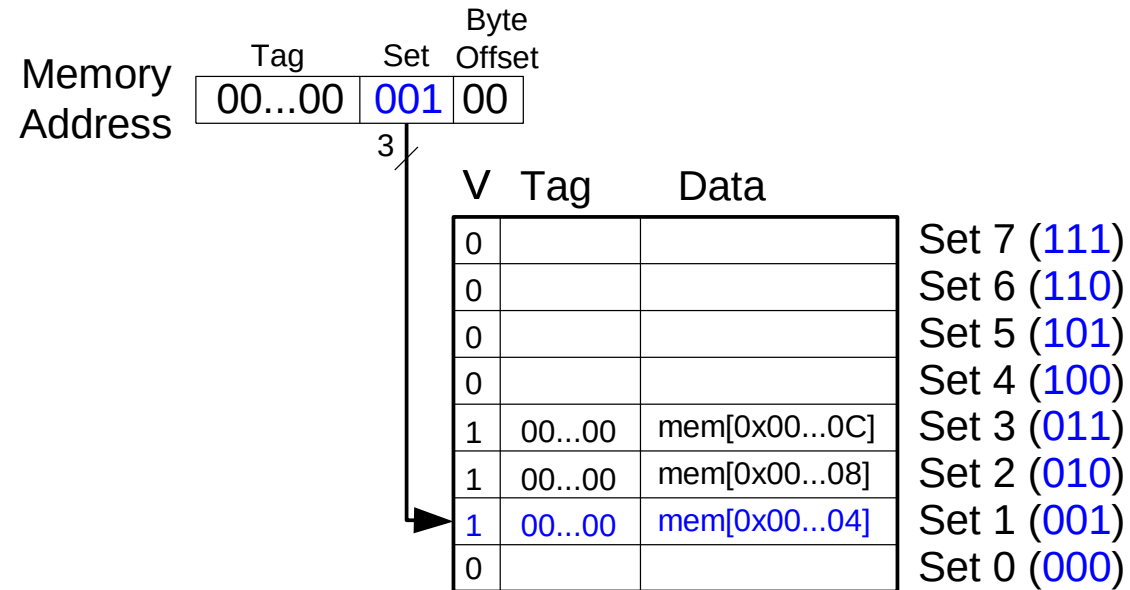
Direct Mapped Cache



Direct Mapped Cache Hardware



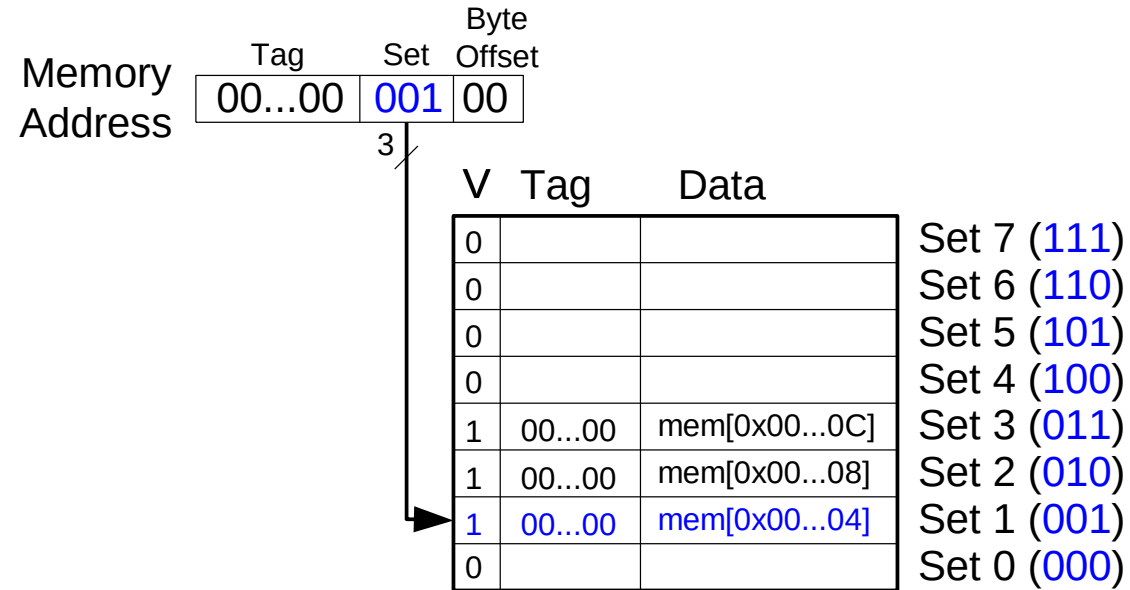
Direct Mapped Cache Performance



```
# MIPS assembly code
loop:    addi $t0, $0, 5
        beq  $t0, $0, done
        lw   $t1, 0x4($0)
        lw   $t2, 0xC($0)
        lw   $t3, 0x8($0)
        addi $t0, $t0, -1
        j    loop
done:
```

Miss Rate =

Direct Mapped Cache Performance



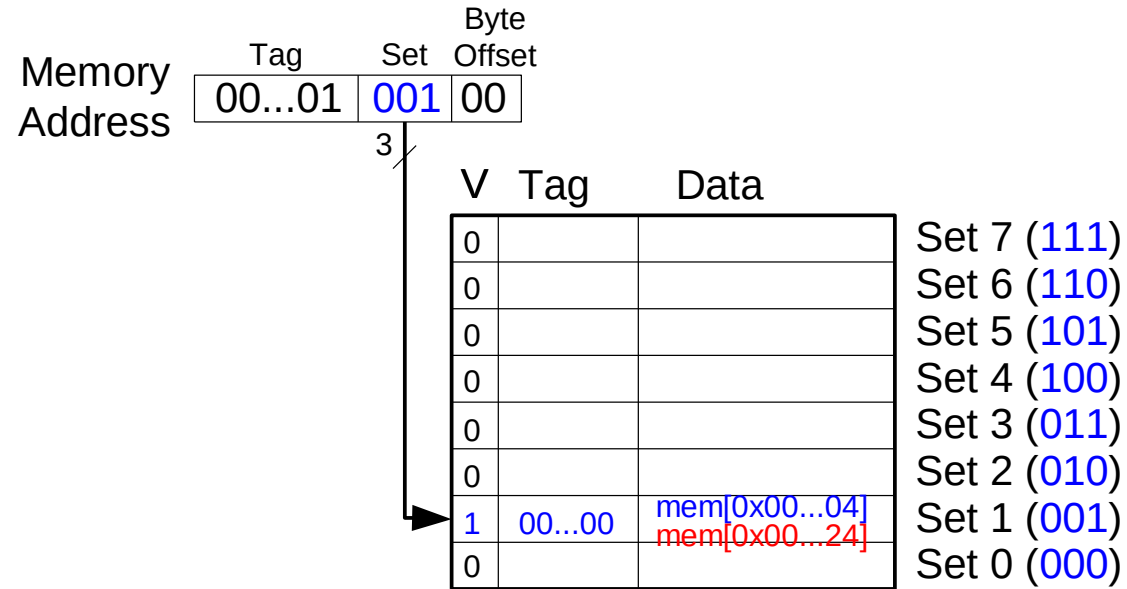
```
# MIPS assembly code
loop:  addi $t0, $0, 5
      beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0xC($0)
      lw   $t3, 0x8($0)
      addi $t0, $t0, -1
      j    loop
done:
```

$$\text{Miss Rate} = 3/15 =$$

20%

Temporal Locality
Compulsory Misses

Direct Mapped Cache: Conflict



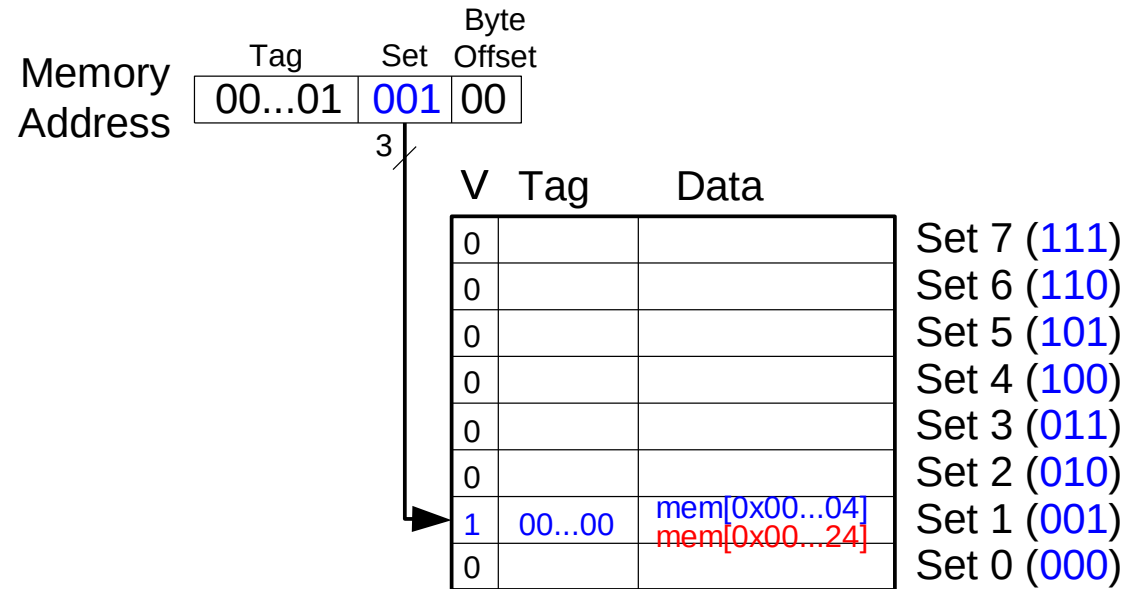
MIPS assembly code

```

    addi $t0, $0, 5
loop: beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0x24($0)
      addi $t0, $t0, -1
      j    loop
done:
    
```

Miss Rate =

Direct Mapped Cache: Conflict

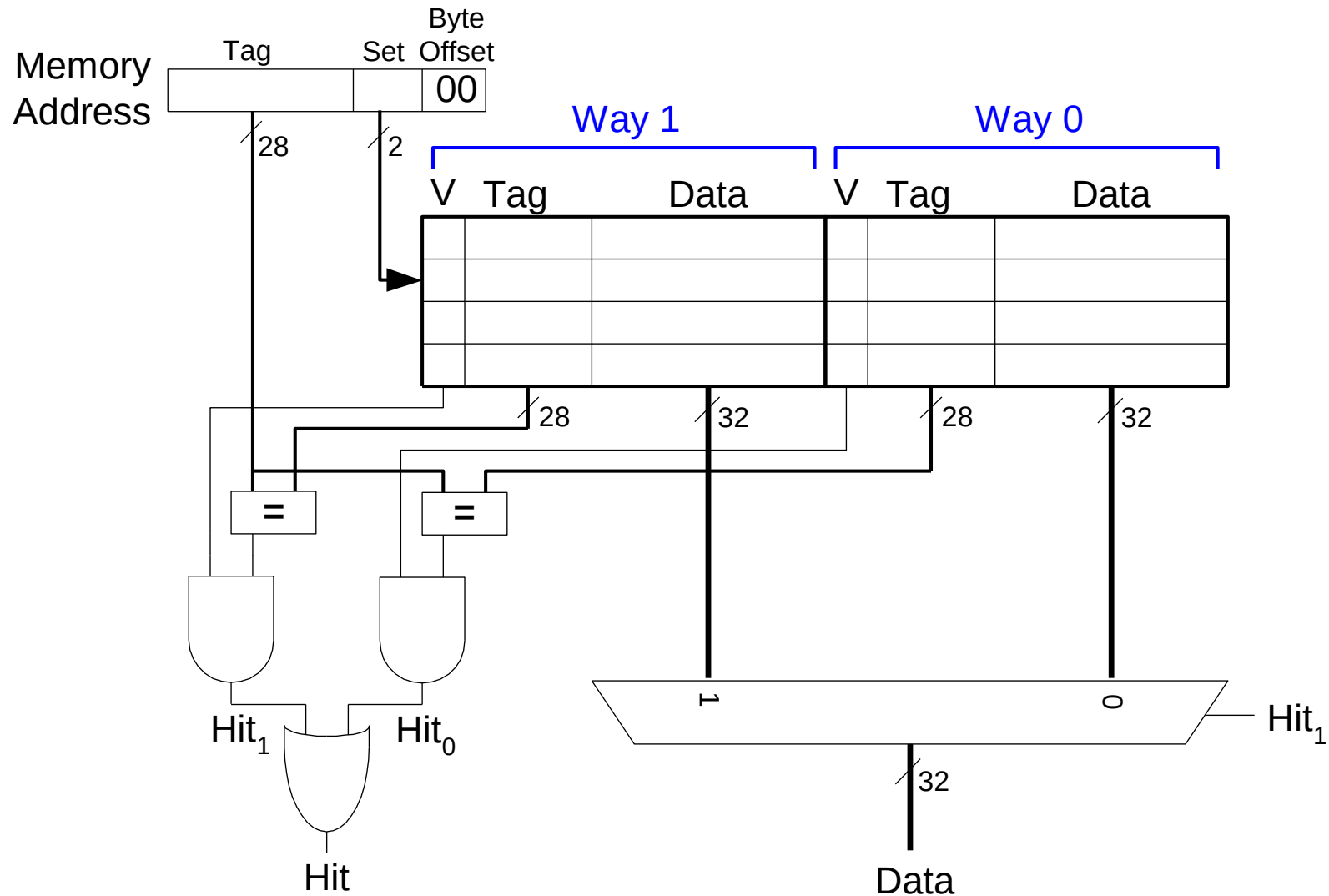


```
# MIPS assembly code
      addi $t0, $0, 5
loop: beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0x24($0)
      addi $t0, $t0, -1
      j    loop
done:
```

*Miss Rate = 10/10
= 100%*

Conflict Misses

N-Way Set Associative Cache



N-way Set Associative Performance

MIPS assembly code

```
loop:  addi $t0, $0, 5
      beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0x24($0)
      addi $t0, $t0, -1
      j    loop
done:
```

Miss Rate =

Way 1			Way 0			
V	Tag	Data	V	Tag	Data	
0			0			Set 3
0			0			Set 2
1	00...10	mem[0x00...24]	1	00...00	mem[0x00...04]	Set 1
0			0			Set 0

N-way Set Associative Performance

MIPS assembly code

```
loop:  addi $t0, $0, 5
      beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0x24($0)
      addi $t0, $t0, -1
      j    loop
done:
```

Miss Rate = 2/10
= 20%

Associativity reduces
conflict misses

Way 1			Way 0			
V	Tag	Data	V	Tag	Data	
0			0			Set 3
0			0			Set 2
1	00...10	mem[0x00...24]	1	00...00	mem[0x00...04]	Set 1
0			0			Set 0

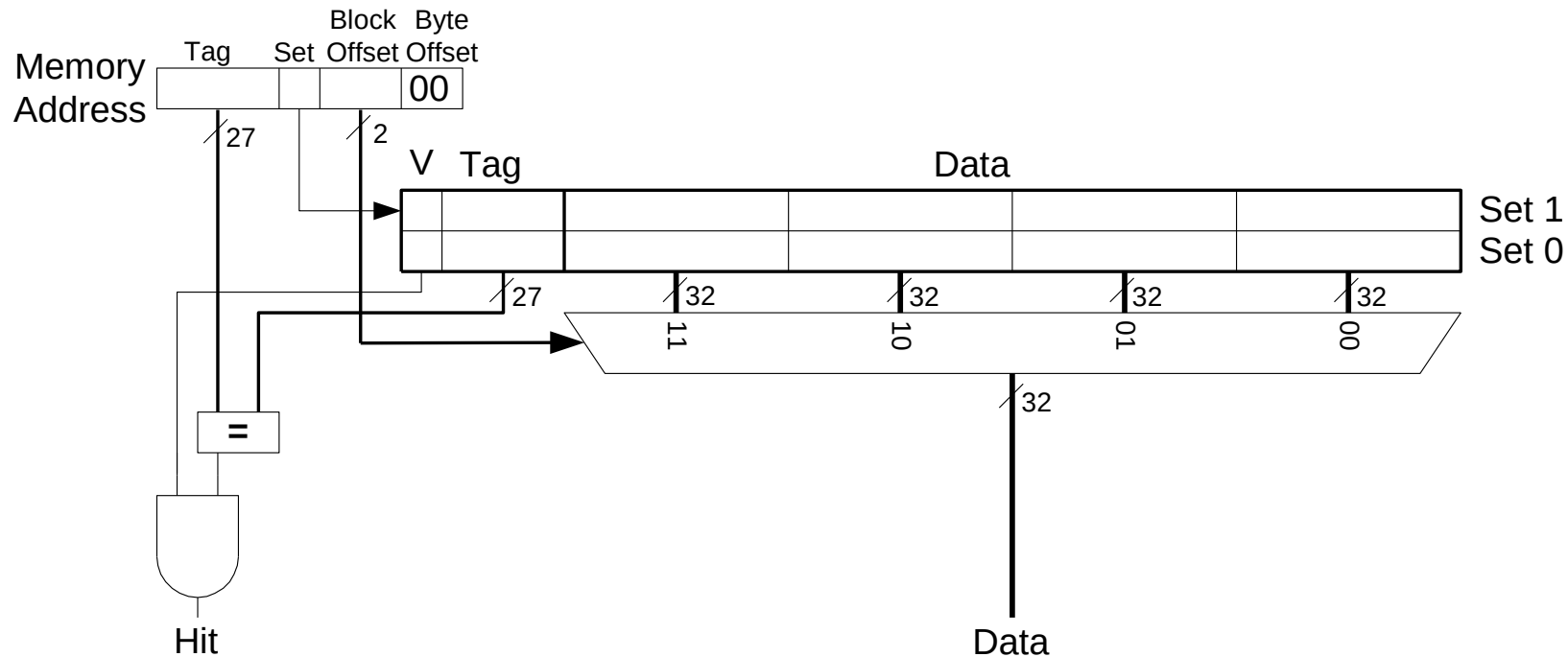
Fully Associative Cache

- No conflict misses
- Expensive to build

[illegible]

Spatial Locality?

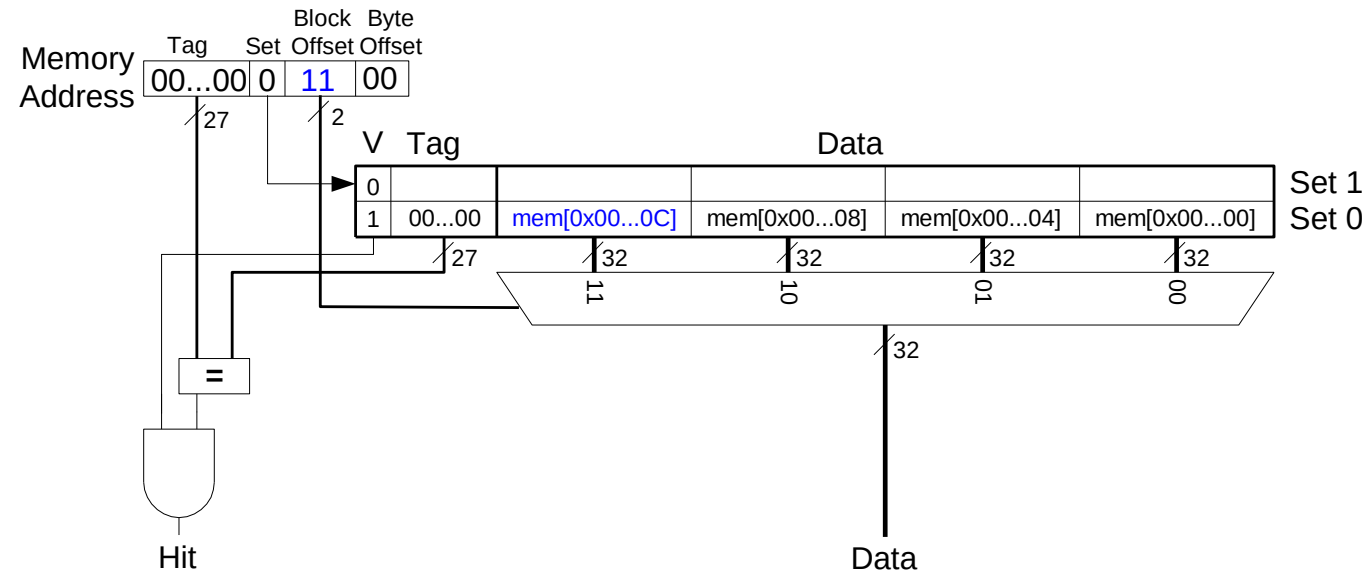
- Increase block size:
 - Block size, $b = 4$ words
 - $C = 8$ words
 - Direct mapped (1 block per set)
 - Number of blocks, $B = C/b = 8/4 = 2$



Direct Mapped Cache Performance

```
      addi $t0, $0, 5
loop: beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0xC($0)
      lw   $t3, 0x8($0)
      addi $t0, $t0, -1
      j    loop
done:
```

Miss Rate =

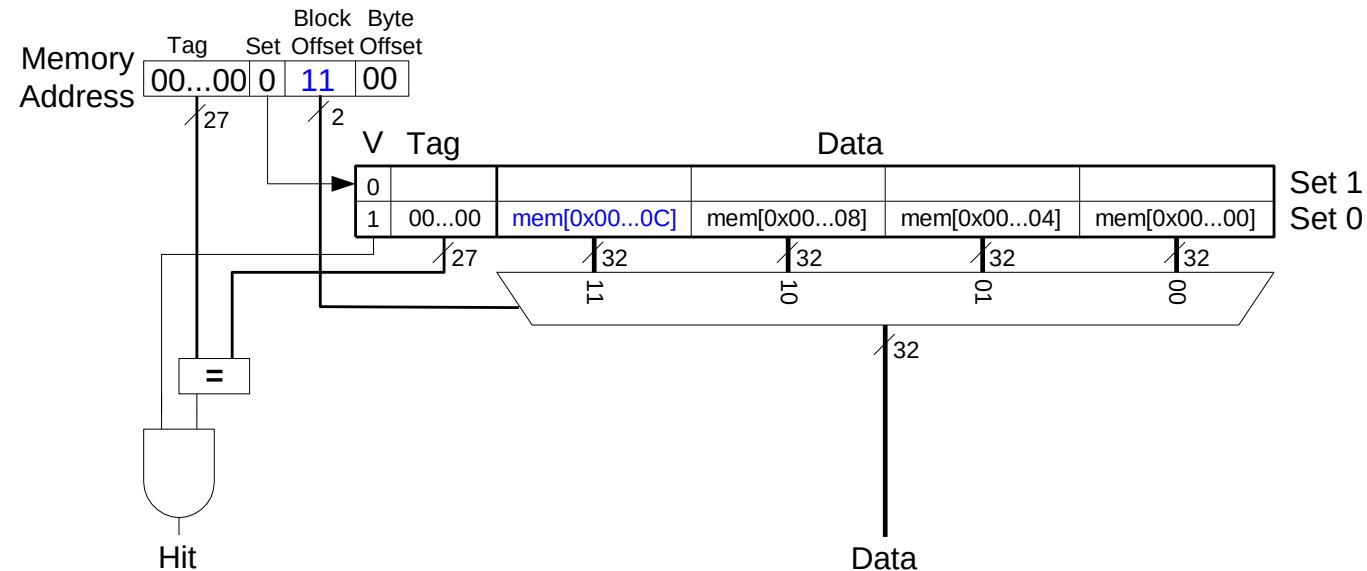


Direct Mapped Cache Performance

```
loop:  addi $t0, $0, 5
      beq  $t0, $0, done
      lw   $t1, 0x4($0)
      lw   $t2, 0xC($0)
      lw   $t3, 0x8($0)
      addi $t0, $t0, -1
      j    loop
done:
```

Miss Rate = 1/15
= 6.67%

Larger blocks reduce compulsory misses through spatial locality



Types of Misses

- **Compulsory**: first time data is accessed
- **Capacity**: cache too small to hold all data of interest
- **Conflict**: data of interest maps to same location in cache
- **Miss penalty**: time it takes to retrieve a block from lower level of hierarchy

Capacity Misses

- Cache is too small to hold all data of interest at one time
 - If the cache is full and program tries to access data X that is not in cache, cache must evict data Y to make room for X
 - **Capacity miss** occurs if program then tries to access Y again
 - X will be placed in a particular set based on its address
- In a **direct mapped** cache, there is only one place to put X
- In an **associative cache**, there are multiple ways where X could go in the set.
- How to choose Y to minimize chance of needing it again?
 - Least recently used (LRU) replacement: the least recently used block in a set is evicted when the cache is full.

Cach block size and cach associativity

Cache block size and cache associativity

- Cache block size (8, 16, 32, 64, or 128 Bytes)↵
- Cache associativity (1, 2, or 4-way)↵

The only statistic that you can collect on this system is cache hit rate after performing a sequence of memory accesses. Here is what you observe: ↵

Addresses Accessed (Oldest -> Youngest): 8 times

Hit Rate↵

31	8192	63	25	16384	4096	8192	64	16384	5/9↵
----	------	----	----	-------	------	------	----	-------	------

Cache block size and cache associativity

- Cache block size (8, 16, 32, 64, or 128 Bytes)↵
- Cache associativity (1, 2, or 4-way)↵

The only statistic that you can collect on this system is cache hit rate after performing a sequence of memory accesses. Here is what you observe: ↵

Addresses Accessed (Oldest -> Youngest): 8 times

Hit Rate↵

31	8192	63	25	16384	4096	8192	64	16384	5/9↵
----	------	----	----	-------	------	------	----	-------	------

What is the size of cache block? Circle one: 8B / 16B / 32B / 64B / 128B↵

What is the associativity of the cache? Circle one: 1-way / 2-way / 4-way↵